The Complete LLM Pipeline

From Raw Data to Production-Grade AI Systems

This guide explains how modern large language models are **actually built, aligned, deployed, and controlled** in the real world.

Not prompt tricks.
Not model hype.
Not abstract research-only theory.

This is **system-level thinking** — the full pipeline from raw data to production behavior.

What this guide covers

- How training data shapes failure modes
- What pretraining really gives you (and what it doesn't)
- When to fine-tune vs when to use RAG (and why hybrids win)
- How preference optimization changes model behavior
- What actually happens at inference time
- How tools, retrieval, memory, and execution fit together
- Where hallucinations come from — and how to engineer against them
- How real systems are deployed, evaluated, monitored, and rolled back

---

Who this is for

- Engineers building real LLM systems
- Builders who want control, not magic
- People shipping products, not demos

If you are looking for shortcuts or vibes, stop here.
If you want **predictable, debuggable, production-grade AI**, keep reading.

Arun
2026

PART 0 — EXECUTIVE SUMMARY & QUICK MAP

0.0 One-line roadmap (Phase 0 → Phase 8)

Collect/clean data → train foundation model → tune model behavior → optimize preferences → control inference → build tooling → adapt for cost/scale → test & monitor → decide trade-offs for deployment.

(This shows progression from raw data to operational system.)

0.1 Condensed training phases (one line each)

0 — **Raw Inputs:** define, collect, clean, govern data. 1 — **Pretrain:** build base model from large corpora. 2 — **Fine-tune:** adapt model to tasks with labeled examples. 3 — **Preference alignment:** reward/policy tune for behavior. 4 — **Runtime:** understand inference choices and limits. 5 — **Tools:** add retrieval, execution, memory systems. 6 — **Deployment:** quantize, serve, automate delivery. 7 — **Evaluation:** benchmarks, failure tracking, drift. 8 — **Trade-offs/Architecture:** justify decisions mathematically.

0.2 How to use this guide

Audience: engineers building or operating generative AI systems; decision-makers choosing architecture strategies. **Checkpoints:**

- After Phase 0 — "Is my data clean, white-listed, free of PII, versioned?"
- After Phase 2 — "Do I have a validated SFT model or do I need RAG?"
- After Phase 5 — "Is integration with tools (retrieval, execution) stable?"

Minimum viable deliverables:

- Versioned dataset with metrics.
- Pretrained backbone or choice of base model.
- Task-tuned model or RAG pipeline.
- Evaluation suite with regression tests.

0.3 Quick decision matrix: when to RAG vs fine-tune vs hybrid

Condition	Use RAG	Use Fine-tune	Hybrid

Condition	Use RAG	Use Fine-tune	Hybrid
Data changes often	Yes (feeds latest info) (Oracle)	No	Yes (fine-tune core domain + RAG dynamic facts) (Medium)
Need up-to-date facts	Yes	No	Yes
Task requires deep domain reasoning	No	Yes	Yes
Low latency, offline	No	Yes	Maybe
Limited labeled data	Yes	No	Yes
High inference throughput	Depends on retrieval performance	Better	Depends

RAG attaches a vector search layer feeding context at query time; fine-tune modifies model weights for task specifics. (IBM)

PART A — ORIENTATION (WHY THIS GUIDE EXISTS)

A.1 Definitions & mental model: model vs system

Model: mathematical function learned from data; predicts next token/label. Not oracle. **System:** model + code + data + toolchain + rules that turn statistical inference into useful behavior. **Statistical learner reality:** outputs are patterns, not guaranteed facts. Hallucination is default error without grounding.

A.2 Risk posture & when NOT to deploy

Do not deploy if:

- Data contains unredacted PII/legal risk.
- No evaluation on safety/edge cases.
- The system influences critical decisions without verification.
- No rollback/monitoring pipeline.

A.3 Regulatory / IP baseline

Licenses: track source licenses; do not use proprietary data without rights. **Provenance:** maintain truth source for every document used in training/RAG. **IP retention:** version datasets with hashes; immutable snapshots.

A.4 How the guide maps to real projects

Checklists for each phase:

- Data: metadata, coverage, bias checks.
- Pretraining: loss curves, tokenization artifacts.
- SFT: label specs, QA checkpoints.
- RAG: index health, retrieval accuracy.
- Evaluation: baseline vs guardrails, drift detectors.

Exit criteria for phases:

- Phase 0 complete when dataset governance and metrics dashboard are established.
- Phase 2 complete when tuned model outperforms base on validation and safety checks.
- Phase 5 complete when tools integrate with traceable logs.

PART 1 — PHASE 0: RAW MATERIAL & CONSTRAINTS (DATA-FIRST)

1.0 Core thesis: data shapes failure modes

Bad data → bad model behavior. Clean, representative, governed data is *the* foundation.

1.1 Data sources and legal flags

Sources: web crawls, curated corpora, domain datasets, synthetic, code. **Legal flags:** copyright, PII, export restrictions, medical/health records. Enforce exclusion lists.

1.2 Data pathologies

Pathologies to detect: noise, duplicates, bias (demographic/topic), stale info, embedded PII.

1.3 Cleaning & pipeline primitives

Deduplication: exact + fuzzy matching. **Language ID:** remove out-of-scope languages. **Length heuristics:** filter overly short/noisy sequences. **PII detection:** regex, ML detectors; redact before use.

1.4 Tokenization considerations

Choices: BPE vs Unigram vs byte-level affect coverage and rare token handling. **Failure cases:** morphologically rich languages, domain symbols.

1.5 Dataset versioning & governance

Record: content hashes, lineage metadata, immutable snapshots. Store with semantic versioning.

1.6 Dataset metrics dashboard

Track: coverage by topic/language, freshness, label distribution, imbalance measures.

1.7 Practical outputs from Phase 0

Deliverables: data spec, exclusion/allow lists, sampling rules, versioned storage.

PART 2 — PHASE 1: PRETRAINING (FOUNDATION MODEL)

2.0 Goals & expected outputs of pretraining

Goal: Learn a base model that captures general language patterns so it can serve as the backbone for many tasks.

What pretraining produces:

- A model checkpoint that has learned token relationships from raw text.
- A vocabulary/tokenizer.
- Training metrics (loss curves, token diversity measures).
- Ability to generate coherent sequences and generalize to new inputs.

Pretrained models are not tuned for specific tasks yet — they are **general-purpose statistical predictors**.

2.1 Corpora construction and sampling strategy

Corpora construction means choosing and combining text sources.

Sources:

- Web text (filtered for quality).
- Books, articles.
- Code repositories.
- Domain-specific texts you care about.

Sampling strategy: Not all data is equally important. Mixing ratios determine how often certain types of data are fed into training.

Guiding rules:

- Diversify sources to avoid domain imbalance.
- Up-sample rare but important categories.
- Divide training data into **time buckets** so newer and older data both appear.
- Avoid overfitting to a narrow genre.

Example logic: for every 1M tokens, you might sample 60% web text, 30% curated books, 10% code — but adjust based on your goal. Advanced research shows careful data mixing can be as impactful as model scaling itself. (GrowAI)

2.3 Training objective variants

Training objective determines what the model tries to predict.

1) Causal LM (autoregressive). Model sees tokens left-to-right and predicts next token. Standard for GPT-like models.

2) Masked LM. Model hides some tokens and predicts them. Used in models like BERT. □ (Wikipedia)

3) Hybrids / blank infilling. Mixes causal and masked tasks to combine strengths.

When to choose which:

- **Causal LM:** best for generation tasks.
- **Masked LM:** strong for representation learning & classification tasks.
- **Hybrid:** when you want both generation and understanding quality.

2.4 Architecture choices

Basic building block: Transformer layers with self-attention.

Key choices:

a) Depth vs width

- Depth = number of layers (higher = more expressiveness).
- Width = layer size/hidden dimension (higher = more capacity). Tradeoff depends on compute budget.

b) Positional encodings Models need to know token order. Methods include learned embeddings or rotary (RoPE). (Preprints)

c) Context window length Longer windows help with long inputs but increase computation (attention cost grows roughly with square of sequence length). (Emergent Mind)

d) Architecture variants Most remain transformer-based, but state-space or sparse attention methods exist too (e.g., sliding window attention). (Sebastian Raschka)

2.5 Optimization & stability

Training large models requires stable optimization.

Common optimizer: AdamW (adaptive gradients + weight decay).

Key techniques:

- **Learning rate schedule:** start small, warm-up phase, plateau, decay.
- **Gradient clipping:** prevents huge updates that destabilize training.
- **Batch sizes:** larger batches stabilize gradients but need more memory.
- **Normalization (LayerNorm):** ensures gradients remain stable.

These ensure training converges without exploding losses or divergence.

2.6 Scaling laws & evidence

Scaling laws describe predictable performance trends when you increase model size, dataset size, or compute.

Chinchilla scaling: shows that for LLMs, both model parameters and amount of training data need to grow in balance for best performance. Too few tokens or too small a model wastes compute. (Wikipedia)

Implications:

- More compute should be split between model size and training tokens.
- You can't just make models bigger and expect linear gains.

These laws guide decisions on how big to make models relative to available data and compute.

2.7 Outputs: base checkpoints, metrics to capture, storage & artifact policy

Base checkpoints:

- Save regular checkpoints during training.
- Tag them with metadata (training step, loss, token throughput).

Metrics to track:

- **Training loss** (cross-entropy).
- **Validation loss** (generalization).
- **Token distribution coverage.**
- **Speed/throughput.**

Storage:

- Immutable artifacts with versioning.
- Keep hashes and metadata records.

Artifact policy:

- Store model weights, tokenizer config, training config, metrics logs.
- Ensure reproducibility by saving random seeds and hyperparameters.

This phase results in a **foundation model** that can be further fine-tuned, adapted, or used in retrieval systems depending on your application needs.

If you want, next output can break down **Phase 2 (Supervised Fine-Tuning)** with clear recipes and checklists.

PART 3 — PHASE 2: SUPERVISED FINE-TUNING (SFT / INSTRUCTION TUNING)

3.0 When to SFT vs when to rely on RAG

Supervised Fine-Tuning (SFT) means updating model weights using labeled examples.

RAG (Retrieval-Augmented Generation) keeps the base model frozen and feeds it retrieved context at inference.

Use **SFT** when:

- Your task needs the model to *write/behave* in a specific way every time.

- You have *good labeled examples* (questions paired with correct outputs).
- You need *consistent domain knowledge* embedded in the model.

Use **RAG** when:

- You have **large dynamic or changing information** (facts that update).
- You lack labeled data but have *documents to search*.
- You want to *avoid retraining costs*.

Hybrid (SFT + RAG) when:

- Core behavior must be tuned, but domain facts are best retrieved at runtime.

3.1 SFT dataset design (schema: system/user/assistant)

SFT data has a **schema** with roles:

Role	Meaning
`system`	Instructions about *how* the assistant should behave
`user`	The user's input/question
`assistant`	The target output the model should generate

Example record (JSON-like):

```
{ system: "Be concise and factual.",
  user: "Explain X in simple steps.",
  assistant: "Step 1: … Step 2: …" }
```

Selection heuristics:

- Include diverse examples covering expected inputs.
- Avoid duplication.
- Cover edge cases (hard questions).
- Balance categories (don't overload one topic).

3.2 Label quality control: annotation specs, QA rules, noise budget

Label quality determines model quality. Low-quality labels → bad fine-tune.

Annotation specs:

- Define exact formatting rules (tone, length limits, no contradictions).

- Provide clear examples of good vs bad answers.

QA rules:

- Use reviewers to score outputs.
- Reject samples outside quality thresholds.

Noise budget:

- Even good datasets contain noise; measure proportion of bad examples tolerated.
- Aim for <5% bad labels in validation set.

3.3 PEFT options (LoRA, prefix tuning) vs full fine-tune — cost/benefit

Full fine-tune

- Updates all model weights.
- Best performance but *expensive* and may overfit.

PEFT (Parameter-Efficient Fine-Tuning) Common methods:

- **LoRA:** Low-rank weight updates added to model weights.
- **Prefix tuning:** Add trainable prefix vectors instead of weights.

Method	Train cost	Model footprint	When best
Full fine-tune	High	Large	When max accuracy matters
LoRA	Low	Small	Most cases with budget limits
Prefix tuning	Low	Small	Better when tuning for multiple behaviors

PEFT lets you store many adapters (one per task) without copying full model.

3.4 Training protocols (LR, epochs, early stopping, catastrophic forgetting)

Learning rate (LR):

- Start small.
- Too high → diverges; too low → no learning.

Epochs:

- Number of times data is seen.
- More epochs → risk of overfitting when dataset is small.

Early stopping:

- Stop training when validation loss stops improving.

Catastrophic forgetting:

- Model forgets pretrained knowledge.
- Mitigate with mixed data (pretraining samples mixed during SFT) or lower LR.

3.5 Evaluation: validation splits, domain holdouts, overfitting checks

Validation splits:

- Split dataset into train/valid/test.
- Train on train, check metrics on valid, final check on test.

Domain holdouts:

- Hold out a whole subset (e.g., all medical Qs) to validate domain generalization.

Overfitting checks:

- If training loss decreases and validation loss increases → overfitting.

Metrics to capture:

- Accuracy (task dependent), BLEU / ROUGE / F1 for text, human eval scores for style.

3.6 Deliverables: instruction-tuned model, adapter registry, validation report

Instruction-tuned model checkpoint

- Trained weights (full or PEFT adapters).
- Metadata: LR, epochs, data version, hyperparameters.

Adapter registry (if using PEFT)

- List of adapters with tags (task, date, data version).

Validation report

- Metrics per split.
- Examples of successes/failures.

- Overfit analysis.

Below is **PART 4 — PHASE 3: PREFERENCE OPTIMIZATION** written to be **conceptually simple, mechanically precise, and system-level**. This phase explains *why* preference optimization exists, *what data it needs*, and *how the algorithms actually work*, without mystification.

PART 4 — PHASE 3: PREFERENCE OPTIMIZATION

(RLHF / DPO / ALTERNATIVES)

4.0 Purpose & when preferences are required

What problem this phase solves

After pretraining + SFT, the model can:

- speak fluently
- follow instructions
- answer many questions

But it still **does not know what humans prefer**.

Preference optimization teaches the model:

- which answers are *better* than others
- how to trade off helpfulness vs safety vs verbosity
- how to avoid answers that are technically correct but useless or unsafe

Key idea: SFT teaches *what to say*. Preference optimization teaches *which of multiple valid answers is better*.

When this phase is required

- User-facing assistants
- Safety-critical domains
- Tasks with subjective quality (tone, usefulness, refusal style)

When you can skip it

- Internal tools
- Deterministic pipelines

- Strictly factual RAG systems with verification

4.1 Preference dataset blueprint

Preference data is **comparative**, not absolute.

Instead of:

> "This answer is correct"

You collect:

> "Answer A is better than Answer B"

Sampling strategy (diversity + tail targeting)

Goals

- Cover normal user cases
- Actively sample edge cases ("tail") where models fail

Sampling rules

- Mix easy, medium, hard prompts
- Include adversarial and ambiguous prompts
- Oversample known failure zones (hallucination, refusal, safety)

Why Models improve fastest where they currently fail.

Annotation instructions (rubrics)

Annotators compare two outputs for the *same prompt*.

They rank based on:

- **Helpfulness**: does it solve the user's problem?
- **Safety**: avoids harmful or disallowed content
- **Specificity**: concrete, not vague
- **Correctness**: factual and logical consistency

Rule Annotators never write answers. They only judge existing ones.

Inter-annotator agreement targets

Agreement measures consistency between annotators.

Metric: Cohen's Kappa

- < 0.4 → instructions unclear
- 0.4–0.6 → acceptable
- > 0.6 → strong agreement

Low agreement means your rubric is broken, not the annotators.

A/B ranking UI & logging format

Each record stores:

- prompt
- answer_A
- answer_B
- chosen_winner
- reason tags (optional)

This data becomes training input for reward learning.

4.2 Reward model design & validation

Reward model (RM) A separate neural network that scores how good an answer is *according to human preferences*.

Input:

- prompt + model answer

Output:

- scalar score (higher = better)

Training

- RM learns to give higher scores to preferred answers

Validation

- Hold out preference pairs
- Check RM agrees with humans on unseen data
- Run adversarial probes (does RM reward verbosity? flattery?)

Reward models fail silently if not validated.

4.3 Optimization algorithms

This is where preference signals modify the main model.

PPO (Proximal Policy Optimization)

How it works

1. Model generates answers
2. Reward model scores them
3. PPO nudges model weights to increase reward
4. KL constraint prevents drifting too far from original model

Pros

- Very flexible
- Strong alignment

Cons

- Complex
- Unstable
- Expensive

DPO (Direct Preference Optimization)

Key simplification No reward model. No reinforcement loop.

Instead:

- Train model directly on preference pairs
- Increase probability of preferred answer over rejected one

Pros

- Simple
- Stable
- Cheaper
- Widely adopted in 2024–2025

Cons

- Less flexible than PPO
- Assumes good preference data

Trade-off table

Method	Complexity	Stability	Cost	When to use
PPO	High	Medium	High	Large labs, complex objectives
DPO	Low	High	Low	Most practical systems
IPO / variants	Medium	Medium	Medium	Research exploration

4.4 Reward hacking detection & mitigation

Reward hacking Model learns to exploit reward signals instead of real quality.

Examples:

- Overly long answers
- Excessive politeness
- Refusal everywhere

Mitigations

- Regular human audits
- Adversarial test prompts
- Penalties for verbosity or repetition
- Periodic retraining of reward model

If you don't look for reward hacking, you will miss it.

4.5 Safety & bias auditing for reward models

Reward models encode human bias.

Audit for:

- demographic bias
- over-refusal
- preference for style over correctness
- suppression of minority viewpoints

Method

- Controlled prompt sets
- Compare score distributions across groups
- Human spot checks

Reward models are policy engines. Treat them as such.

4.6 Outputs of Phase 3

Aligned policy checkpoint

- Main model after preference optimization

Preference dataset artifact

- Versioned
- Audited
- Linked to rubric version

Supporting artifacts

- Reward model (if PPO)
- Training configs
- Evaluation results

Phase 3 exit condition

You exit this phase only when:

- Model preferences match human expectations
- Safety failures are reduced
- No obvious reward hacking remains

PART 5 — PHASE 4: RUNTIME MECHANICS & LIMITS

5.0 Inference loop and decoding strategies

What “inference” means

Inference is the act of **using a trained model to generate output**.

Basic inference loop

1. Take user input
2. Convert text → tokens
3. Run model forward pass

4. Choose next token
5. Repeat until stop condition

The critical part is **step 4: how you choose the next token**.

Decoding strategies (how the model picks tokens)

The model outputs probabilities for the next token. Decoding decides how to sample from them.

Greedy decoding

- Always pick the highest-probability token
- Deterministic
- Often repetitive and boring

Use when:

- You want strict reproducibility
- Code generation with constraints

Top-k sampling

- Only consider top *k* most likely tokens
- Sample randomly among them

Effect:

- More diversity than greedy
- Still controlled

Nucleus (top-p) sampling

- Consider the smallest set of tokens whose probabilities sum to *p* (e.g. 0.9)
- Dynamic cutoff

Effect:

- Better balance between coherence and creativity
- Industry default for chat models

Temperature

- Scales randomness
- Low temperature → safer, repetitive
- High temperature → creative, risky

Rule

- Temperature controls *how random*
- Top-k / top-p control *where randomness is allowed*

5.1 Context window realities and long-context mitigation

What the context window is

The context window is the **maximum number of tokens the model can see at once**.

Includes:

- System prompt
- Conversation history
- Retrieved documents
- User input

When it fills up, something must be dropped.

Hard limits

- Attention cost grows roughly with the square of context length
- Longer context = slower + more expensive

Common long-context mitigation patterns

Sliding window

- Keep only recent messages

Summarization

- Compress older context into a summary

Chunked retrieval (RAG)

- Only inject relevant spans, not full documents

Hierarchical prompting

- One model summarizes → another answers

Key idea: More context ≠ better answers if it's noisy.

5.2 Stateless APIs vs persistent memory design

Stateless APIs

Each request is independent.

Pros:

- Simple
- Scales easily
- No privacy risk from stored memory

Cons:

- No long-term personalization

Used by:

- Most production APIs

Persistent memory systems

Store information across sessions.

Types:

- User profiles
- Conversation summaries
- Task state

Risks:

- Privacy leakage
- Memory pollution
- Stale or incorrect memories

Rule Memory must be:

- Explicit
- Reviewable
- Evictable

Never let models write memory blindly.

5.3 Hallucination taxonomy and root causes

Hallucinations are not one thing. They have different causes.

Fabrication

Model invents facts.

Cause:

- Training prior outweighs evidence
- No grounding signal

Misattribution

Correct fact, wrong source.

Cause:

- Weak citation logic
- RAG errors

Temporal hallucination

Outdated information presented as current.

Cause:

- Pretraining cutoff
- Missing retrieval

Logical hallucination

Steps don’t follow logically.

Cause:

- Long reasoning chains
- Weak internal consistency

RAG-induced hallucination

Model uses retrieved text incorrectly.

Cause:

- Irrelevant chunks
- Overconfidence in retrieval

Important: RAG can **reduce** hallucinations, but also **create new ones** if badly designed.

5.4 Determinism, reproducibility, and seed handling

Determinism

Same input → same output.

Requires:

- Fixed decoding parameters
- Fixed random seed
- Same model version
- Same hardware behavior (not always guaranteed)

Why determinism matters

- Debugging
- Audits
- Regression testing
- Legal/safety review

Seeds

Random seeds control sampling randomness.

Rules:

- Log seeds in production
- Use fixed seeds for eval
- Never rely on “default randomness” for critical systems

5.5 Practical runtime instrumentation

If you don’t measure it, you don’t control it.

Latency SLOs

Track:

- Time to first token
- Tokens per second
- P95 / P99 latency

Token accounting

Track:

- Input tokens
- Output tokens
- Retrieval tokens

Used for:

- Cost control
- Abuse detection
- Capacity planning

Token traces

Log:

- Prompt
- Retrieved context
- Generated output
- Model parameters used

Purpose:

- Debug failures
- Explain behavior
- Support audits

Phase 4 exit condition

You exit this phase only when:

- Runtime behavior is predictable
- Latency and cost are controlled
- Hallucination failure modes are understood
- Outputs are traceable and reproducible

PART 6 — PHASE 5: TOOL AUGMENTATION

(RAG, EXECUTION, MEMORY)

6.0 Tool taxonomy and orchestration patterns

What a “tool” is

A tool is **anything the model can call that is not itself**.

Examples:

- Search or retrieval system
- Code execution engine
- Database
- Calculator
- Memory store
- API call

The model **decides when to use a tool**, but **does not execute it directly**. Your system executes the tool and feeds results back.

Tool taxonomy (types)

1. Knowledge tools

- Search
- Retrieval (RAG)
- Databases

2. Action tools

- Code execution
- Calculators
- File operations
- External APIs

3. Memory tools

- User profile store
- Conversation summaries
- Task state

Orchestration patterns (how tools are wired)

Single-step

- Model → tool → answer

Multi-step

- Model → tool → model → tool → answer

Guarded

- Tool output validated before model sees it

Rule The more powerful the tool, the stricter the guard.

6.1 RAG implementation blueprint

RAG = Retrieval-Augmented Generation Meaning: *retrieve first, then generate.*

Embedding model choices & vector DB ops

Embeddings

- Turn text into vectors (numbers)
- Similar meaning → similar vectors

Choose embedding models based on:

- Domain match
- Vector size
- Cost

Vector database operations

- Insert (index documents)
- Search (top-k nearest vectors)
- Filter (by metadata like date, source)

Chunking strategy & span granularity

Documents must be split into chunks.

Too small

- Loss of context

Too large

- Retrieval noise
- Context window waste

Rules of thumb

- Chunk by meaning, not fixed length
- Overlap chunks slightly
- Keep chunks independently understandable

Retrieval reranking & relevance metrics

Initial vector search is approximate.

Reranking

- Take top-k retrieved chunks
- Use a stronger model to reorder by relevance

Metrics

- Recall: did we retrieve the right info?
- Precision: is retrieved info actually useful?

Bad retrieval = confident hallucinations.

6.2 RAG SECURITY & PROVENANCE CHECKLIST

RAG adds power **and risk**.

Source whitelisting & trust tiers

Not all sources are equal.

Trust tiers

- Tier 1: verified internal docs

- Tier 2: reviewed external sources
- Tier 3: unverified / user-provided

Only allow high-trust sources for critical answers.

Chunk signing & provenance tokens

Each chunk should carry:

- Source ID
- Timestamp
- Hash
- Trust tier

This allows:

- Auditing
- Rollbacks
- Claim verification

Span-level redaction & access control

Before retrieval:

- Remove PII
- Enforce user permissions

Never rely on the model to “ignore” sensitive data.

Citation mapping & source→claim verification

Process

1. Retrieve chunks
2. Generate answer
3. Map each claim to supporting chunk
4. Reject unsupported claims

This is the only reliable way to reduce hallucinations in RAG.

6.3 Code execution & calculators

(sandboxing & verification loop)

Why code execution exists

Models are bad at:

- Exact arithmetic
- Long calculations
- State tracking

Let code do what code is good at.

Sandboxing

Code must run in:

- Isolated environment
- No network unless allowed
- Resource limits

Never run raw model-generated code on production machines.

Verification loop

1. Model proposes code
2. System runs code
3. Output validated
4. Result fed back to model

If code fails, model retries with error message.

6.4 External memory systems

Memory ≠ context window.

Memory schema

Memory should be structured:

- key
- value
- timestamp
- confidence
- source

Avoid free-text memory blobs.

Retrieval policies

Rules decide:

- When memory is read
- When memory is written
- When memory is ignored

Memory should be:

- Explicit
- Reviewable
- Deletable

Eviction strategies

Memory must expire.

Strategies:

- Time-based
- Relevance-based
- Confidence-based

Unbounded memory leads to corrupted behavior.

6.5 Orchestration frameworks and audit trails

Orchestration frameworks

Frameworks help wire:

- Models
- Tools
- Memory
- Logging

Examples:

- Off-the-shelf frameworks
- Custom pipelines (often safer for production)

Framework choice matters less than **traceability**.

Audit trails

Every interaction should log:

- Input
- Tools called
- Tool outputs
- Model outputs
- Parameters used

Purpose:

- Debugging
- Compliance
- Safety investigations

If you cannot reconstruct a bad answer, you cannot fix the system.

Phase 5 exit condition

You exit this phase only when:

- Tools are gated and auditable
- RAG sources are trusted and traceable
- Code execution is sandboxed
- Memory is controlled and reversible

PART 7 — PHASE 6: PRACTICAL ADAPTATION

(PEFT, QUANTIZATION, DEPLOY)

7.0 Why adapt, not retrain

Retraining means:

- Re-running pretraining or full fine-tuning
- High compute cost
- Long iteration cycles
- High risk of regressions

Adaptation means:

- Reusing a trained model
- Making small, controlled changes
- Optimizing for cost, speed, and deployment constraints

Core rule

> If the base model already "knows language," do not retrain it. Adapt it.

Most production systems fail not because the model is weak, but because **serving it is too expensive or unstable**.

7.1 LoRA / QLoRA / adapters — when & how

These are **parameter-efficient fine-tuning (PEFT)** methods.

They let you adapt behavior without touching most model weights.

What LoRA is (simple view)

Instead of updating big weight matrices:

- Add small trainable matrices
- Learn only those
- Keep base model frozen

Result:

- 1–5% of parameters trained
- Much cheaper
- Easy rollback

QLoRA

QLoRA = LoRA + quantized base model.

- Base model stored in low precision (e.g. 4-bit)
- LoRA layers trained in higher precision

Effect:

- Massive memory savings
- Allows fine-tuning large models on smaller GPUs

When to use which

Scenario	Recommended
Limited GPU memory	QLoRA
Many task variants	LoRA adapters
One critical task	LoRA or full FT
Frequent updates	LoRA

Step-by-step adaptation recipe (conceptual)

1. Load base model (frozen)
2. Attach adapter layers
3. Train only adapters
4. Save adapters separately
5. Swap adapters at runtime if needed

Adapters turn one base model into **many behaviors**.

7.2 Quantization strategies (4-bit, GPTQ)

Quantization reduces numeric precision of weights.

Why quantize

- Smaller model size
- Faster inference
- Lower memory bandwidth
- Lower cost per request

Common strategies

8-bit

- Minimal accuracy loss
- Good default

4-bit

- Major memory savings
- Some quality loss
- Often acceptable for chat / RAG

GPTQ / similar

- Post-training quantization
- No retraining required

Fidelity vs latency trade-off

Precision	Speed	Memory	Quality
FP16	Medium	High	Best
8-bit	Fast	Medium	Near-best
4-bit	Fastest	Low	Slight drop

Rule:

> Quantize until quality degrades beyond acceptable limits — not further.

7.3 Serving stack options and cost models

Serving = running inference for users.

What matters in serving

- Throughput (tokens/sec)
- Latency (time to first token)
- Memory usage
- GPU utilization
- Stability under load

Common serving patterns

Batching

- Combine requests
- Improves GPU efficiency
- Increases latency if uncontrolled

Paged / continuous attention

- Better memory reuse
- Enables long context serving

Cost model basics

Total cost depends on:

- Model size
- Precision
- Tokens per request
- Requests per second
- Hardware type

Always estimate **cost per 1k tokens** before deploying.

7.4 CI/CD for model artifacts

Models are artifacts. Treat them like code.

What goes through CI/CD

- Model weights / adapters
- Tokenizer
- Prompts
- Decoding parameters
- Retrieval configs

Canary deployments

- New model serves small % of traffic
- Compare metrics against baseline
- Roll forward or rollback

Shadow serving

- New model runs in parallel
- Users see old output
- Logs used for evaluation

This catches failures before users do.

7.5 Operational runbook

Production systems fail. Plan for it.

Rollback

- One-command revert to previous model
- No retraining
- No manual patching

If rollback takes hours, it's broken.

Emergency kill-switch

Immediate actions:

- Disable tool access
- Force safe decoding
- Switch to static responses if needed

Never rely on the model to self-disable.

Incident playbooks

Pre-define responses for:

- Hallucination spikes
- Latency explosions
- Cost overruns
- Data leaks

Write these **before** incidents occur.

Phase 6 exit condition

You exit this phase only when:

- Model can be served cheaply
- Latency and cost are predictable
- Updates are reversible
- Failures are survivable

PART 8 — PHASE 7: EVALUATION, FAILURE MODES & CONTROL

8.0 Evaluation framing: distributional metrics, tail risk, human-in-the-loop

Why evaluation is hard

Models don't fail evenly. They work **most of the time** and fail **rarely but badly**.

So evaluation must measure:

- **Average quality**
- **Worst-case behavior**
- **Human trust impact**

Distributional metrics

Do not look only at averages.

Track:

- Mean performance
- Worst-10% cases
- Long-tail errors

A model that is "95% correct" can still be unusable if the 5% causes harm.

Tail risk

Tail risk = rare but severe failures.

Examples:

- Confident hallucinations
- Unsafe completions
- Wrong medical or legal advice

These are not caught by normal benchmarks.

Human-in-the-loop

Some judgments cannot be automated.

Humans are required for:

- Safety review
- Tone and usefulness

- Ambiguous edge cases

Automation assists humans; it never replaces them here.

8.1 Recommended benchmarks and where to place them

What benchmarks are good for

Benchmarks test **capabilities**, not **trustworthiness**.

Use them to:

- Compare model versions
- Detect regressions
- Measure known skills

Types of benchmarks

General reasoning

- Reading comprehension
- Logical consistency
- Multi-step reasoning

Truthfulness

- Measures tendency to hallucinate
- Penalizes confident false answers

Domain benchmarks

- Medical QA
- Legal QA
- Code correctness

Where benchmarks fit in the pipeline

Benchmarks should be run:

- After pretraining (baseline)
- After SFT
- After preference optimization
- Before deployment
- Periodically in production (sampled)

Never use benchmarks as the final approval gate.

8.2 Reproducible experiment templates

Evaluation must be **repeatable**.

Seed control

Randomness affects results.

Rules:

- Fix seeds for evaluation
- Log all seeds used
- Never mix random and fixed runs

Config logging

Every experiment must log:

- Model version
- Dataset version
- Prompt templates
- Decoding parameters
- Tool settings

If it’s not logged, it didn’t happen.

Dataset splits

Use:

- Train
- Validation
- Test
- Domain holdouts

Never tune on test data.

Statistical tests

Use simple tests:

- Bootstrap confidence intervals
- Paired comparisons between models

Do not trust single runs.

8.3 Human evaluation playbook

Human eval is structured, not ad-hoc.

Annotation guides

Annotators must know:

- What “good” means
- What “bad” means
- When to abstain

Provide concrete examples.

Agreement auditing

Track how often annotators agree.

Low agreement means:

- Rubric unclear
- Task ill-defined

Fix the rubric before collecting more data.

Grading rubric

Score separately:

- Correctness
- Helpfulness
- Safety
- Clarity

Never collapse everything into one score.

8.4 Model-based judges

Models can evaluate other models.

When to use

- Large-scale regression testing
- Fast iteration cycles

Rules for safe use

- Periodically calibrate against humans
- Never use as final authority
- Track drift over time

Model judges drift silently if unchecked.

8.5 Hallucination mitigation patterns

Hallucinations must be **engineered against**, not hoped away.

Retrieve → generate → verify

1. Retrieve evidence
2. Generate answer
3. Verify claims against sources
4. Reject unsupported output

Citation enforcement

Require:

- Each factual claim has a source
- No source → abstain or hedge

Abstention rules

Teach the system:

- When not to answer
- How to say “I don’t know”

Abstention is a success case, not a failure.

8.6 Confidence & uncertainty quantification

Models always sound confident. That confidence is **not real**.

Temperature scaling

Lower temperature:

- Less randomness
- More conservative answers

Higher temperature:

- More creativity
- More risk

Bayesian ensembling

Run multiple models or seeds.

If they disagree:

- Confidence is low
- Trigger verification or abstention

Conformal prediction (simple view)

Set a threshold:

- Below threshold → answer
- Above threshold → abstain

Threshold tied to retrieval strength and agreement.

8.7 Adversarial testing

You must attack your own system.

Prompt injection

Test:

- “Ignore previous instructions”
- Role confusion
- Hidden instructions in documents

Context poisoning

Test:

- Malicious retrieved content
- Conflicting sources

Long-context stress tests

Test:

- Very long prompts
- Irrelevant noise
- Reordered context

Lineage attacks

Test:

- Fake citations
- Source spoofing

If attackers can break it, they will.

8.8 Continuous monitoring

Evaluation never ends.

Drift detection

Monitor:

- Input distribution changes
- Output style shifts
- Retrieval relevance decay

Canary prompts

Keep a fixed set of prompts. Run them daily.

Any change = investigation.

Automated rollback triggers

Define thresholds:

- Hallucination rate
- Latency spikes
- Cost explosions

Cross threshold → rollback automatically.

8.9 Cost / latency / quality trade-off guide

Everything trades off.

Typical tensions

- Higher quality → higher cost
- Lower latency → lower context
- More safety → more refusals

Decision rule

Optimize for:

1. Safety
2. Predictability
3. Cost
4. Quality

Never reverse this order in production systems.

Phase 7 exit condition

You exit this phase only when:

- Failures are measurable
- Rare risks are understood
- Rollbacks are automatic

- Humans remain in control

PART 9 — PHASE 8: DESIGN TRADE-OFFS & ARCHITECTURE JUSTIFICATION

9.0 Weight-centric vs system-centric decision framework

Weight-centric thinking (model-first)

Focuses on:

- Bigger models
- Better fine-tuning
- Smarter alignment

Assumption:

> "If the model is good enough, the system will work."

Reality:

- Expensive
- Slow to iterate
- Fragile under distribution shift

System-centric thinking (pipeline-first)

Focuses on:

- Retrieval
- Verification
- Guardrails
- Tooling
- Monitoring

Assumption:

> "The model is one component inside a controlled machine."

Reality:

- Cheaper
- More robust

- Easier to debug and audit

Decision rule

If a failure can be fixed **without touching weights**, do not touch weights.

Weights are the last lever, not the first.

9.1 RAG vs Fine-Tuning vs Hybrid decision rubric

This decision should be mechanical, not emotional.

Use **RAG** when:

- Knowledge changes frequently
- Answers must be sourced
- Hallucination risk is high
- Data is large but unlabeled

Failure mode:

- Retrieval errors
- Overconfidence in bad sources

Use **Fine-Tuning** when:

- Behavior/style must be consistent
- Task is stable
- Latency must be low
- No external data is required

Failure mode:

- Stale knowledge
- Overfitting
- Costly retraining

Use **Hybrid** when:

- Behavior is stable
- Knowledge is dynamic
- Safety matters

- You need both control and freshness

Hybrid is the default for serious systems.

9.2 Single-pass vs multi-pass generation

Single-pass generation

One prompt → one answer.

Pros:

- Fast
- Cheap
- Simple

Cons:

- No verification
- No correction
- High hallucination risk

Use only for:

- Low-risk tasks
- Drafting
- Internal tooling

Multi-pass generation

Generate → check → refine.

Examples:

- Retrieve → generate → verify
- Plan → execute → review
- Answer → critique → revise

Pros:

- Safer
- More accurate
- More controllable

Cons:

- Higher latency
- Higher cost

Use for:

- User-facing
- Safety-critical
- High-impact decisions

Rule

If a wrong answer is expensive, single-pass is unacceptable.

9.3 Agentic vs deterministic pipeline rules

Agentic systems

Model decides:

- What to do
- Which tool to call
- In what order

Pros:

- Flexible
- Powerful

Cons:

- Hard to predict
- Hard to audit
- Easy to jailbreak

Deterministic pipelines

System decides:

- Fixed steps
- Explicit tool calls
- Clear boundaries

Pros:

- Predictable

- Auditable
- Safer

Cons:

- Less flexible

Hybrid rule

Let the model **suggest**, never **execute blindly**.

The system must always be the final authority.

9.4 Explicit system boundaries & claims

Every system must answer two questions **in writing**.

What the system guarantees

Examples:

- Sources are cited
- Unsafe queries are refused
- Calculations are verified
- Memory can be deleted

Guarantees must be:

- Testable
- Enforced
- Auditable

What the system does NOT guarantee

Examples:

- Perfect correctness
- Up-to-date facts without retrieval
- Legal/medical advice
- Zero hallucinations

Stating non-guarantees is a safety feature, not a weakness.

Rule

If you cannot explain the boundary, the boundary does not exist.

9.5 Governance, auditability, and compliance mapping

Governance

Who is responsible when the system fails?

Define:

- Owner
- Escalation path
- Kill-switch authority

Without ownership, incidents repeat.

Auditability

You must be able to answer:

- Why did this answer happen?
- Which data was used?
- Which model version?
- Which tools were called?

If the answer is “we don’t know”, the system is unsafe.

Compliance mapping

Map system components to requirements:

- Data sources → licenses
- Memory → privacy rules
- Logs → retention policy
- Models → version control

Compliance is not paperwork. It is architecture.

Phase 8 exit condition (final)

You are done **only when**:

- Every major design choice is explicit
- Trade-offs are written down
- Failure modes are acknowledged
- Rollbacks are possible
- Responsibility is assigned

At this point, you no longer have “a model”.

You have a **system that can survive reality**.

This completes the full guide.